ML MINI PROJECT · FINAL YEAR B.E.

# Titanic Survival Prediction

Your mini project explained section by section, with the results interpreted and probable viva / oral questions with answers


| | | |
|---|---|---|
| 1 | **The project at a glance** | Part 1 |
| 2 | **The dataset and what the charts show** | Part 2 |
| 3 | **Pre-processing and feature engineering explained** | Part 3 |
| 4 | **Random Forest explained** | Part 4 |
| 5 | **Evaluation and your result** | Part 5 |
| 6 | **Probable viva questions with answers** | Part 6 |


Based on your report **PCCOER_Titanic_Survival_Prediction_Mini_Project_Report.pdf** (Random Forest, Kaggle train.csv).

How to use: read the theory once, trace the worked example by hand, then cover the answers and test yourself on the questions. Answers are written the way you would say them in a viva — short first line, then the detail if the examiner digs deeper.

PART 1

# The project at a glance

**Title:** Titanic Dataset Survival Prediction (Machine Learning mini project)
**Team:** Yash Dhadge (BECOA41), Sarvesh Dharmadhikari (BECOA45), Divyesh Ravane (BECOA62), Vaishnav Pawar (BECOA61)

## Problem statement

Build a machine learning model that predicts which kinds of passengers survived the Titanic shipwreck, using passenger data such as name, age, gender and socio-economic class. Dataset: Kaggle "Titanic: Machine Learning from Disaster" ( `train.csv` ).

## What you did, step by step

| Step | What was done | Result |
|---|---|---|
| 1. Load | `pd.read_csv("train (1).csv")` | 891 passengers × 12 columns |
| 2. Explore | age histogram, class bar chart, survival pie, gender pie | most aged 20–40; 3rd class largest (491); 38.4% survived; 64.8% male |
| 3. Clean & select | drop Name, Cabin, Ticket, PassengerId; one-hot encode Sex and Embarked ( `drop_first=True` ); fill missing with the column mean | 8 features: Pclass, Age, SibSp, Parch, Fare, Sex_male, Embarked_Q, Embarked_S |
| 4. Split | `train_test_split(test_size=0.2, random_state=42)` | 712 train, 179 test |
| 5. Train | `RandomForestClassifier(n_estimators=100, random_state=42)` | 100 decision trees |
| 6. Evaluate | `accuracy_score(y_test, y_pred)` | **0.8045 → about 80.4%** (144 of 179 correct) |

**ONE-MINUTE ANSWER TO "EXPLAIN YOUR PROJECT"**

"We used the Kaggle Titanic dataset of 891 passengers to predict survival. After exploring the data with Matplotlib charts, we dropped identifier-like columns such as Name, Ticket, Cabin and PassengerId, one-hot encoded Sex and Embarked, and filled missing ages with the mean. That gave eight numeric features. We split the data 80/20 and trained a Random Forest of 100 decision trees, which predicted survival correctly for about 80.4% of the 179 unseen test passengers. It is a supervised binary classification problem."

## Type of problem

**Supervised learning → binary classification.** Each passenger has a known label ( `Survived` : 1 = survived, 0 = did not), and the model learns to predict that label from the other columns.

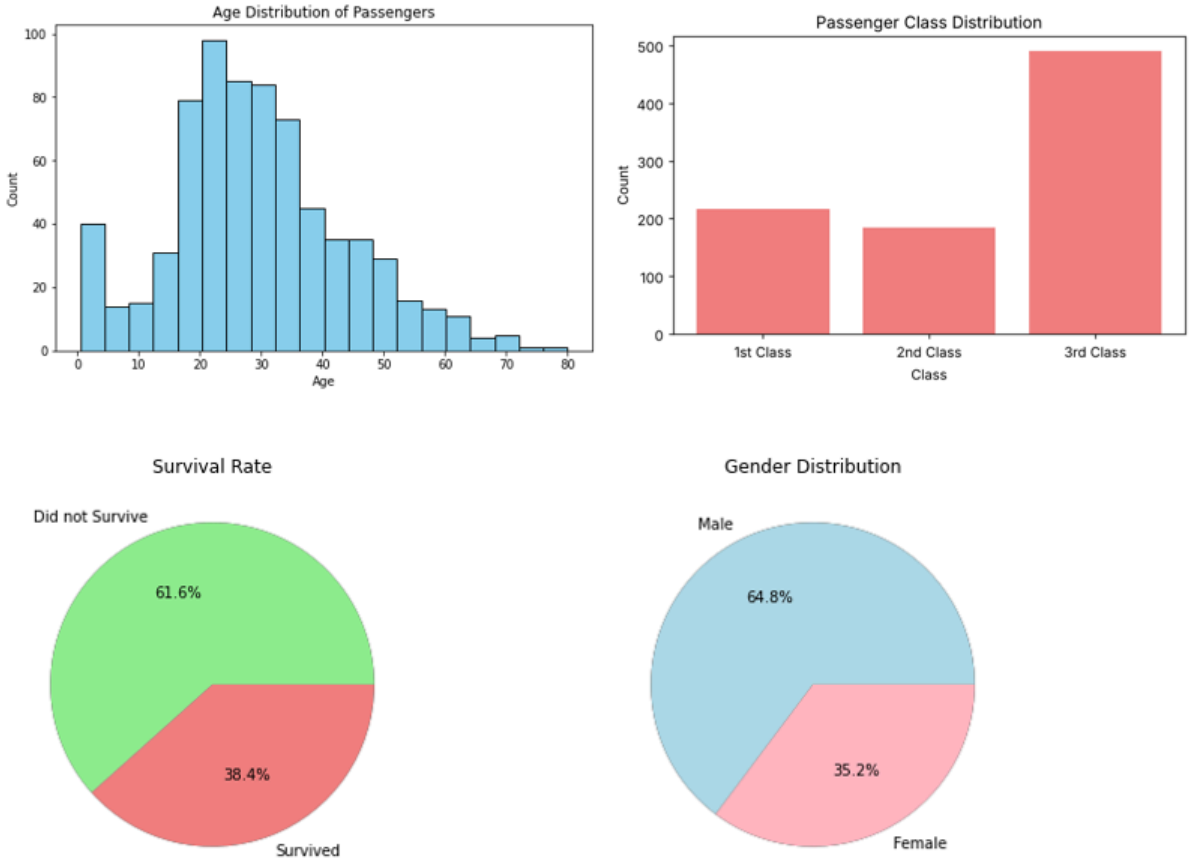
PART 2

# The dataset and what the charts show

## Columns of train.csv

| Column | Type | Meaning | Used in model? |
|---|---|---|---|
| PassengerId | ID | row number | no — carries no information |
| **Survived** | target | 0 / 1 | **target y** |
| Pclass | ordinal | ticket class 1, 2, 3 (proxy for wealth) | yes |
| Name | text | full name with title (Mr, Mrs, Miss, Master) | no (dropped) |
| Sex | categorical | male / female | yes → `Sex_male` |
| Age | numeric | years (177 missing) | yes, mean-filled |
| SibSp | numeric | siblings + spouses aboard | yes |
| Parch | numeric | parents + children aboard | yes |
| Ticket | text | ticket number | no (dropped) |
| Fare | numeric | price paid | yes |
| Cabin | text | cabin number (687 of 891 missing) | no (dropped) |
| Embarked | categorical | C = Cherbourg, Q = Queenstown, S = Southampton (2 missing) | yes → `Embarked_Q`, `Embarked_S` |

## Your exploratory charts




Your four EDA figures: age histogram, class distribution, survival rate and gender distribution.

- **Age:** most passengers were 20–40; a separate bump of young children (0–5). Slightly right-skewed.
- **Class:** 3rd class 491, 1st 216, 2nd 184 — more than half travelled 3rd class.
- **Survival:** 38.4% survived (342), 61.6% did not (549) — the classes are **moderately imbalanced**.
- **Gender:** 64.8% male (577), 35.2% female (314).

**Well-known patterns in this dataset** (useful when the examiner asks "what did you learn from the data?"): about **74% of women survived but only about 19% of men**; survival was about **63% in 1st class, 47% in 2nd and 24% in 3rd** — the "women and children first" rule and better access to lifeboats for upper decks. A trivial rule "every woman survives, every man dies" is already right for about **78.7%** of the 891 passengers, which is why `Sex` is by far the strongest feature.

PART 3

# Pre-processing and feature engineering explained

## Dropping columns

- **PassengerId** — just a counter; any pattern would be coincidence.
- **Name** — unique text; cannot be used directly (though the *title* inside it is useful — see improvements).
- **Ticket** — messy alphanumeric codes, almost unique per passenger.
- **Cabin** — 77% missing.

## Encoding categorical variables

Models need numbers. Two common methods:

| Method | How | When |
|---|---|---|
| **Label encoding** | each category → an integer (male 1, female 0) | binary or **ordered** categories |
| **One-hot encoding** | one 0/1 column per category | **unordered** categories with 3+ values |

Your code: `pd.get_dummies(data, columns=['Sex', 'Embarked'], drop_first=True)`.

- `Sex` → **Sex_male** (1 = male, 0 = female).
- `Embarked` → **Embarked_Q** and **Embarked_S**; Cherbourg is the dropped baseline (both 0).
- **Why `drop_first=True`?** The last column can always be inferred from the others (they sum to 1). Keeping all creates perfect multicollinearity — the **dummy variable trap**. It matters for linear models; tree models are not harmed either way, but it removes redundancy.

Example — a passenger who boarded at Q: Embarked_Q = 1, Embarked_S = 0. At C: both 0.

## Handling missing values

`data.fillna(data.mean())` replaces each missing value with its column's mean — in practice it fills the 177 missing **Age** values with the average age (about 29.7 years).

- **Mean** is simple but pulled by outliers; **median** is more robust for skewed data; better still, fill Age with the median of passengers with the same **title** or **class**.
- The 2 missing `Embarked` values become 0 in both dummy columns, so they are silently treated as Cherbourg.
- **Data leakage note:** the mean was computed on the full dataset **before** the train/test split, so test passengers' ages slightly influenced the training data. The correct way is to compute the mean on the training set only (or use a `Pipeline` with `SimpleImputer`). The effect here is small, but examiners like this question.

## Feature engineering (proposed, not yet in the code)

| New feature | How | Why it helps |
|---|---|---|
| **FamilySize** | SibSp + Parch + 1 | very small and very large families survived less |
| **IsAlone** | FamilySize == 1 | lone travellers (mostly men) had lower survival |
| **Title** | extract Mr / Mrs / Miss / Master from Name | captures sex, age group and status in one feature |
| **AgeBand / FareBand** | bin continuous values | reduces noise, helps simple models |
| **HasCabin** | Cabin not missing | cabin known ≈ richer passenger |

PART 4

# Random Forest explained

## Decision tree first

A decision tree asks a sequence of yes/no questions — "Sex_male ≤ 0.5?", "Pclass ≤ 2.5?", "Age ≤ 6.5?" — and each leaf predicts the majority class of the training passengers that reach it. At each node the tree chooses the split that makes the child nodes **purest**.

**Gini impurity:** Gini = $1 - \Sigma\, p_i^2$ (0 = pure, 0.5 = 50/50 for two classes). **Entropy:** $-\Sigma\, p_i \log_2 p_i$. scikit-learn uses Gini by default.

*Worked example* — a node with 10 passengers, 4 survived: Gini = $1 - (0.4^2 + 0.6^2)$ = **0.48**. Split by sex: 4 females (3 survived) → Gini = $1 - (0.75^2 + 0.25^2)$ = 0.375; 6 males (1 survived) → Gini = $1 - ((1/6)^2 + (5/6)^2)$ = 0.278. Weighted Gini = 0.4 × 0.375 + 0.6 × 0.278 = **0.317** → the split reduces impurity by 0.163, so it is a good question.

A single deep tree memorises the training data (**overfitting / high variance**).

## From one tree to a forest

1. **Bootstrap sampling (bagging):** each of the 100 trees is trained on a random sample of the 712 training rows drawn **with replacement** — some rows appear twice, about 37% are left out of each tree.
2. **Random feature subsets:** at each split a tree considers only a random subset of features (√8 ≈ 2–3 features for classification), so trees are different from each other.
3. **Voting:** each tree predicts 0 or 1; the forest outputs the **majority vote** (scikit-learn actually averages the trees' class probabilities).

Averaging many decorrelated trees keeps their low bias but cancels much of their variance → better accuracy on unseen data than one tree.

| Hyperparameter | Your value | Meaning |
|---|---|---|
| `n_estimators` | 100 | number of trees |
| `random_state` | 42 | repeatable bootstraps and feature choices |
| `max_depth` | None (default) | trees grow until leaves are pure — can be tuned to reduce overfitting |
| `max_features` | "sqrt" (default) | features considered at each split |
| `criterion` | "gini" (default) | impurity measure |

**Why Random Forest suits this data:** handles mixed numeric and encoded categorical features, needs **no feature scaling**, captures interactions (a young boy in 2nd class vs an adult man in 3rd), is robust to outliers in Fare, and gives **feature importances**.

PART 5

# Evaluation and your result

## What 80.4% means

`accuracy_score` = correct predictions / total = 144 / 179 = **0.8045**. The model labelled about 4 out of 5 unseen passengers correctly.

Put it in context:

| Model | Approximate accuracy |
|---|---|
| Predict "died" for everyone (majority class) | ≈ 62% |
| "Women survive, men die" rule | ≈ 79% (on the full training file) |
| **Your Random Forest** | **80.4%** on the test split |

So the forest beats the trivial baseline clearly, and slightly beats the gender rule — most of its power comes from `Sex`, with Pclass, Age and Fare adding a little more.

## Metrics you should be able to explain (the report lists them as next steps)

| | Predicted died (0) | Predicted survived (1) |
|---|---|---|
| **Actually died (0)** | TN | FP |
| **Actually survived (1)** | FN | TP |

| Metric | Formula | Meaning here |
|---|---|---|
| Accuracy | (TP + TN) / total | share of passengers labelled correctly |
| Precision | TP / (TP + FP) | of those predicted to survive, how many did |
| Recall | TP / (TP + FN) | of the real survivors, how many we found |
| F1-score | 2PR / (P + R) | balance of precision and recall |

Code to add for the full report:

```
from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=["Died", "Survived"]))
print(sorted(zip(model.feature_importances_, X.columns), reverse=True))
```

## Limitations and improvements

- Only **accuracy** reported; add confusion matrix, precision, recall, F1 and **k-fold cross-validation** (one 179-row test split is noisy — a different `random_state` can move accuracy by a few percent).
- **Stratify** the split (`stratify=y`) to keep 38% survivors in both sets.
- Impute **after** splitting (avoid leakage); use median/title-based Age imputation.
- Engineer **Title, FamilySize, IsAlone**.
- Tune hyperparameters (`max_depth`, `min_samples_leaf`, `n_estimators`) with **GridSearchCV**.
- Compare with Logistic Regression, SVM, KNN, Gradient Boosting / XGBoost.

- **Deployment:** save with `joblib.dump(model, "titanic_rf.pkl")`, load in a Flask/Streamlit app, apply the same pre-processing to new passenger input, return the prediction.

PART 6

# Probable viva questions with answers

## About the project

**Q1. What is the aim of your project?**

To predict whether a Titanic passenger survived, from features like class, sex, age, family aboard, fare and port of embarkation, using a Random Forest classifier — and to practise the full ML workflow from raw data to evaluation.

**Q2. Which dataset did you use and how big is it?**

The Kaggle "Titanic: Machine Learning from Disaster" training file `train.csv`: 891 passengers, 12 columns, with the survival label.

**Q3. Is this supervised or unsupervised? Classification or regression?**

Supervised, because each passenger has a known label. Binary classification, because the target has two classes: survived (1) or not (0).

**Q4. What is the target variable and what are the features?**

Target: `Survived`. Features (8): Pclass, Age, SibSp, Parch, Fare, Sex_male, Embarked_Q, Embarked_S.

**Q5. What accuracy did you get? Is it good?**

About 80.4% (144 of 179 test passengers). It is clearly better than predicting the majority class (~62%) and slightly better than the simple "women survive" rule (~79%). For this dataset, 78–84% is typical for good models.

**Q6. What did the exploratory analysis show?**

Most passengers were 20–40 years old, 3rd class was the largest group (491), only 38.4% survived, and 64.8% were male. In this dataset women and 1st-class passengers had much higher survival rates.

## Data pre-processing

**Q7. Why did you drop Name, Ticket, Cabin and PassengerId?**

PassengerId is just an index; Name and Ticket are almost unique text values that the model cannot use directly; Cabin is about 77% missing. Keeping them would add noise or need heavy processing.

**Q8. Which columns had missing values and how did you handle them?**

Age (177), Cabin (687) and Embarked (2). Cabin was dropped; Age was filled with the column mean through `fillna(data.mean())`; the two missing Embarked values became zeros in both dummy columns.

**Q9. Mean vs median imputation — which is better here?**

Median is more robust when the data is skewed or has outliers. For Age, an even better approach is the median within groups (by title or class), because a "Master" (boy) and a "Mrs" have very different typical ages.

**Q10. Is there any data leakage in your code?**

A small one: the mean used for filling was calculated on the whole dataset before splitting, so test rows influenced it. The correct practice is to fit the imputer on the training set and apply it to the test set, ideally with a Pipeline.

**Q11. What is one-hot encoding? Why use it for Embarked?**

Creating a separate 0/1 column for each category. Embarked (C, Q, S) has no natural order; encoding it as 0, 1, 2 would wrongly imply S > Q > C.

**Q12. What does `drop_first=True` do and why?**

It drops one dummy column per feature (female for Sex, C for Embarked), which becomes the baseline. This avoids the dummy variable trap — redundant columns that always sum to 1.

**Q13. What is the difference between label encoding and one-hot encoding?**

Label encoding maps categories to integers in one column and suits binary or ordered categories. One-hot encoding creates one binary column per category and suits unordered categories.

**Q14. Did you scale the features? Why or why not?**

No. Random Forest splits on thresholds of one feature at a time, so scaling does not change the result. Scaling would be needed for KNN, SVM, logistic regression with regularisation or neural networks.

**Q15. What is feature engineering? Give examples for Titanic.**

Creating more informative features from existing ones: FamilySize = SibSp + Parch + 1, IsAlone, Title from Name (Mr, Mrs, Miss, Master), age and fare bands, HasCabin.

**Q16. What is feature selection? How would you do it?**

Keeping only useful features. Methods: domain knowledge (what we did), correlation analysis, Random Forest feature importances, recursive feature elimination, or chi-square tests.

## Random Forest

**Q17. What is a decision tree?**

A flowchart-like model that splits data with feature-based questions at each node; each leaf gives a class. Splits are chosen to make child nodes as pure as possible.

**Q18. What is Gini impurity? Calculate it for a node with 4 survivors out of 10.**

Gini = $1 - \Sigma p_i^2$. For 4/10 survived: $1 - (0.4^2 + 0.6^2) = 1 - (0.16 + 0.36) = 0.48$. A pure node has Gini 0.

**Q19. What is entropy and information gain?**

Entropy = $-\Sigma\ p_i \log_2 p_i$ measures disorder. Information gain = parent entropy − weighted average entropy of the children. The split with the highest gain is chosen (used in ID3/C4.5).

**Q20. What is a Random Forest?**

An ensemble of decision trees, each trained on a bootstrap sample of the data and using random feature subsets at each split. The final prediction is the majority vote (or averaged probability) of all trees.

**Q21. Why is a Random Forest better than a single decision tree?**

A single deep tree overfits — it has high variance. Averaging many different trees reduces variance while keeping low bias, so the forest generalises better to new passengers.

**Q22. What is bagging?**

Bootstrap aggregating: train several models on random samples drawn with replacement from the training data and combine their predictions by voting or averaging.

**Q23. What is the out-of-bag (OOB) score?**

Each tree leaves out about 37% of rows from its bootstrap sample. Predicting those left-out rows with the trees that did not see them gives a free validation score (`oob_score=True`).

**Q24. What does `n_estimators=100` mean? What happens if you increase it?**

The forest has 100 trees. More trees make predictions more stable but slower; accuracy usually levels off after a few hundred trees. More trees do not cause overfitting.

**Q25. Which features are most important for survival?**

Sex is by far the strongest. Fare, Age and Pclass usually follow. You can show this with `model.feature_importances_`.

**Q26. Bagging vs boosting?**

Bagging trains models independently in parallel and averages them (reduces variance) — Random Forest. Boosting trains models one after another, each focusing on the previous model's mistakes (reduces bias) — AdaBoost, Gradient Boosting, XGBoost.

**Q27. What are the disadvantages of Random Forest?**

Less interpretable than one tree, slower and larger in memory, and can still overfit noisy data if trees are very deep. It cannot extrapolate beyond the range of training values.

## Evaluation

**Q28. Why split the data into training and testing sets?**

To check how well the model works on passengers it has never seen. Accuracy on training data would reward memorisation.

**Q29. What does `random_state=42` do?**

Fixes the random shuffle (and the forest's randomness), so results are the same every run and can be reproduced.

**Q30. Why might accuracy alone be misleading?**

With 62% non-survivors, a model predicting "died" for everyone gets about 62% accuracy while finding no survivors. Precision, recall and F1 show how well each class is handled.

**Q31. Define precision and recall for this problem.**

Precision: of passengers the model predicted would survive, the fraction who really survived. Recall: of all real survivors, the fraction the model identified.

**Q32. What is a confusion matrix?**

A 2 × 2 table of actual vs predicted classes: TN, FP, FN, TP. All classification metrics are computed from it.

**Q33. What is cross-validation and why would it help here?**

Splitting data into k folds, training on k − 1 and testing on the remaining fold, repeated k times and averaged. With only 179 test rows a single split is noisy; 5- or 10-fold CV gives a more reliable accuracy estimate.

**Q34. What is overfitting? How would you detect it in your model?**

The model learns noise in the training data and does worse on new data. Compare training accuracy with test (or cross-validation) accuracy — a Random Forest with unlimited depth often has ~98% training accuracy against ~80% test accuracy, which signals some overfitting.

**Q35. Why did you not use `stratify=y` ?**

It was not used in the report; it would be an improvement. Stratifying keeps the 38/62 survivor ratio identical in training and test sets, making the evaluation more representative.

## Tools and deployment

**Q36. Which libraries did you use and for what?**

pandas — loading, cleaning, encoding, imputing; scikit-learn — train_test_split, RandomForestClassifier, accuracy_score; Matplotlib — histogram, bar and pie charts; Jupyter Notebook — interactive environment.

**Q37. How would you deploy this model?**

Save it with joblib or pickle, load it in a web app (Flask, FastAPI or Streamlit) where a user enters passenger details, apply exactly the same pre-processing (dummy columns, mean age), and return the predicted survival with its probability.

**Q38. How would you predict for the Kaggle test.csv file?**

Apply identical pre-processing (drop the same columns, same dummies, fill with the *training* means — note test.csv also has one missing Fare), align the columns with the training features, call `model.predict` , and save PassengerId with the prediction as the submission CSV.

**Q39. What did you learn from this project?**

The complete ML workflow: exploring and cleaning messy real data, encoding categories, handling missing values, training an ensemble model, evaluating it honestly, and the importance of avoiding leakage and using more than one metric.